

CodeKids — Blueprint Document

Volumen V: Configuración de Infraestructura como Código mediante
render.yaml

MÓDULO TÉCNICO

Orquestación Cluster & Blueprints

DESARROLLADORES CLAVE

Alan / Aquino / Francisco / Alonso

FASE DE DESPLIEGUE

Producción Contingente / Entorno Web

FECHA DE SISTEMA

Mayo 2026

1. Introducción a Render Blueprints

Para asegurar que el despliegue de CodeKids en Render sea consistente, replicable y libre de errores de configuración manual derivados de la interfaz gráfica, se ha implementado la especificación de **Infraestructura como Código (IaC)** a través de un archivo de especificación de Blueprint denominado `render.yaml` situado en la raíz del proyecto.

Este enfoque permite mapear de manera declarativa los servicios web estáticos, las dependencias de construcción y las reglas de ruteo interno. Al subir este manifiesto al repositorio, Render lee su estructura de forma automática y levanta los contenedores y redes distribuidas (CDN) sin intervención manual del equipo de ingeniería.

2. Especificación Completa del Archivo `render.yaml`

A continuación se expone la estructura íntegra y sin restricciones del archivo de manifiesto `render.yaml` optimizado para la arquitectura estática y modular de la aplicación web CodeKids:

```

services:
  - type: web
    name: codekids-platform-frontend
    env: static
    buildCommand: "npm run build"
    publishDir: "./dist"
    pullRequestPreviewsEnabled: true
    autoDeploy: true
    routes:
      - type: rewrite
        source: "/Vistas_Publicas/*"
        destination: "/Vistas_Publicas/*"
      - type: rewrite
        source: "/Dashboard/*"
        destination: "/Dashboard/*"
      - type: rewrite
        source: "/"
        destination: "/index.html"
    headers:
      - path: "/"
        name: X-Frame-Options
        value: DENY
      - path: "/"
        name: X-Content-Type-Options
        value: nosniff
      - path: "/"
        name: Content-Security-Policy
        value: "default-src 'self' https://*.firebaseio.com https://*.googleapis.com;
script-src 'self' 'unsafe-inline' https://cdn.jsdelivr.net; style-src 'self' 'unsafe-
inline' https://fonts.googleapis.com https://use.fontawesome.com; font-src 'self'
https://fonts.gstatic.com https://use.fontawesome.com; img-src 'self' data: https://
*.googleapis.com https://*.unpkg.com;"

```

3. Análisis Detallado de Propiedades y Directivas

3.1 Tipo de Servicio y Entorno de Ejecución

La propiedad `type: web` combinada con `env: static` indica a la infraestructura de Render que el sitio no requiere un servidor HTTP activo basado en Node.js o Python en tiempo de ejecución de producción. En su lugar, Render compila el código y monta los activos resultantes directamente en su red perimetral global optimizada (CDN), reduciendo el tiempo de latencia y los costos operativos al mínimo.

3.2 Comandos de Compilación y Directorio de Publicación

El parámetro `buildCommand` ejecuta las tareas de preparación de activos del frontend de CodeKids. Aunque es una aplicación basada en Vanilla JS, este comando se encarga de llamar al compilador de

Tailwind CSS para procesar las clases utilitarias utilizadas en las interfaces de tres columnas y purgar el CSS no utilizado. El `publishDir: "./dist"` le define a los balanceadores de carga de Render qué carpeta específica contiene el punto de entrada indexado definitivo.

Importancia del Ruteo de Contingencia (Rewrite Rules)

Dado que la plataforma maneja enrutamiento avanzado e interceptores en `auth.js` para restringir accesos según roles (Administrador, Profesor, Estudiante), las reglas de reescritura de rutas son críticas. La regla `source: "/*"` hacia `destination: "/index.html"` asegura que si un usuario refresca el navegador dentro de un subdirectorio protegido del Dashboard, Render no devuelva un error HTTP 404, sino que transfiera el control de rutas al motor interno JS.